Introduction to Artificial Intelligence

Lecture 3: Games and Adversarial search

Prof. Gilles Louppe
g.louppe@uliege.be


PRINCETON
1927
BIG THREE
CHAMPIONS

Today

> How to act rationally in a **multi-agent** environment? How to anticipate and respond to the **arbitrary behavior** of other agents?

- Adversarial search
  - Minimax
  - α-β pruning
  - H-Minimax
  - Expectiminimax
  - Monte Carlo Tree Search
- Modeling assumptions
- State-of-the-art agents.

Games

Formal definition

A **game** is defined by

- the **states** $s \in \mathcal{S}$ and the **initial state** $s_1 \in \mathcal{S}$;
- the **actions** $\mathrm{actions}(s) \subseteq \mathcal{A}$ available in state s;
- the **transition model** $s' = \mathrm{result}(s, a)$, the state reached by taking a in s;
- a function $\mathrm{player}(s) \in \{1, \ldots, N\}$ that says whose turn it is in state s;
- the **terminal states** $T \subseteq \mathcal{S}$, tested by $\mathrm{terminal\text{-}test}(s)$, i.e. $s \in T$;
- a **utility** $\mathrm{utility}(s, p)$, the payoff of player p when the game ends in $s \in T$, e.g. 1, 0 or $\frac{1}{2}$ for a win, a loss or a draw.

Together, s_1, $\mathrm{actions}$ and result define the **game tree**, whose nodes are states and whose edges are actions.

Strategies

Even in a deterministic and fully observable environment, the opponents may act **arbitrarily**: the next state depends on their choices, which we cannot predict.

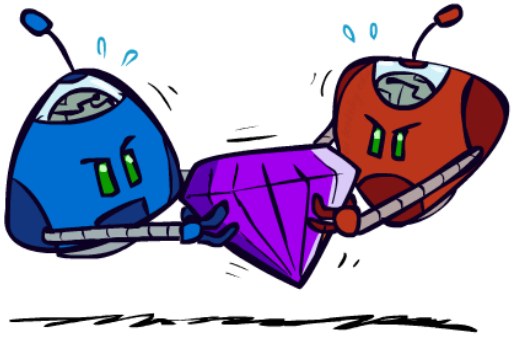
A fixed sequence of actions $a_{1:T}$, the solution of a search problem, is then of no use, since the opponent may lead the game anywhere. A **solution** is instead a **strategy**, or **policy**, $\pi_p : \mathcal{S} \to \mathcal{A}$ for player p: a move for every state the opponent can lead to.

Zero-sum games

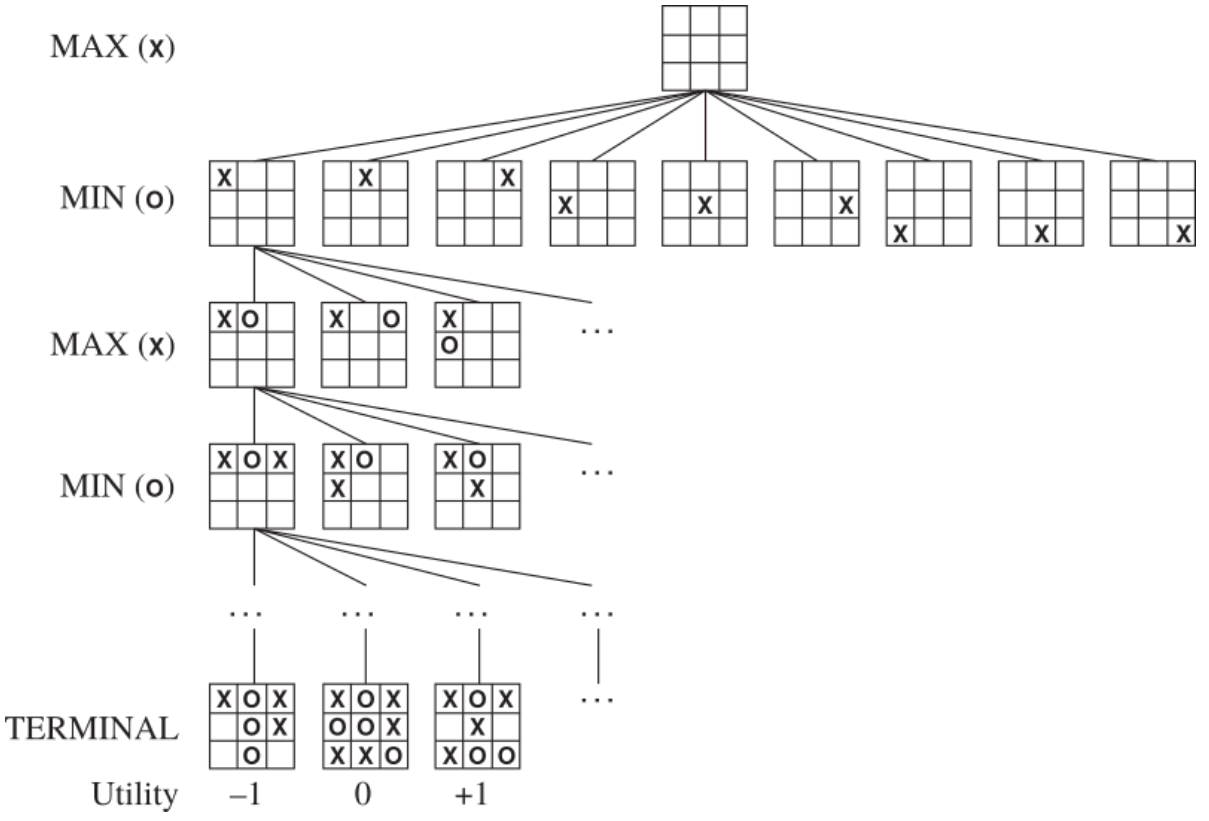
A game is **zero-sum** if the utilities of the players sum to the same constant c in every terminal state:

$$\sum_{p=1}^{N} \text{utility}(s, p) = c \quad \text{for all } s \in T.$$

- e.g., in chess, $c = 1$: $1 + 0$, $0 + 1$ or $\frac{1}{2} + \frac{1}{2}$.
- With two players, $\text{utility}(s, 2) = c - \text{utility}(s, 1)$: a **single utility** describes the game, which player 1 **maximizes** and player 2 **minimizes**.
- This is **strict competition**: what one player gains, the other loses.



Tic-Tac-Toe game tree



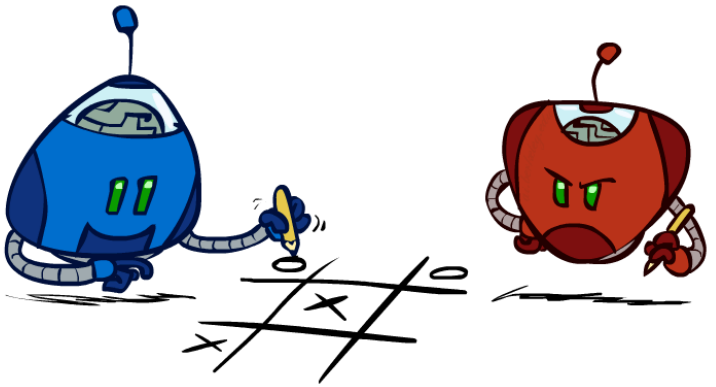
What is an optimal strategy (or perfect play)? How do we find it?

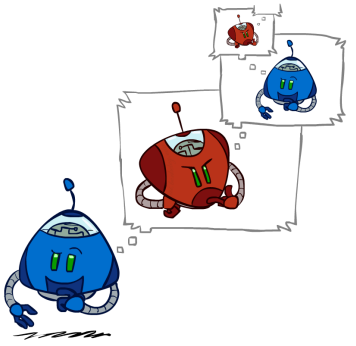
Minimax

Assumptions

Games vary along three axes: **chance**, **information** and the **number of players**. We start with the simplest case of each.

- We assume a **deterministic**, **turn-taking**, **two-player zero-sum game** with **perfect information**.
    - e.g., Tic-Tac-Toe, Chess, Checkers, Go, etc.
- We will call our two players MAX and MIN. MAX moves first.



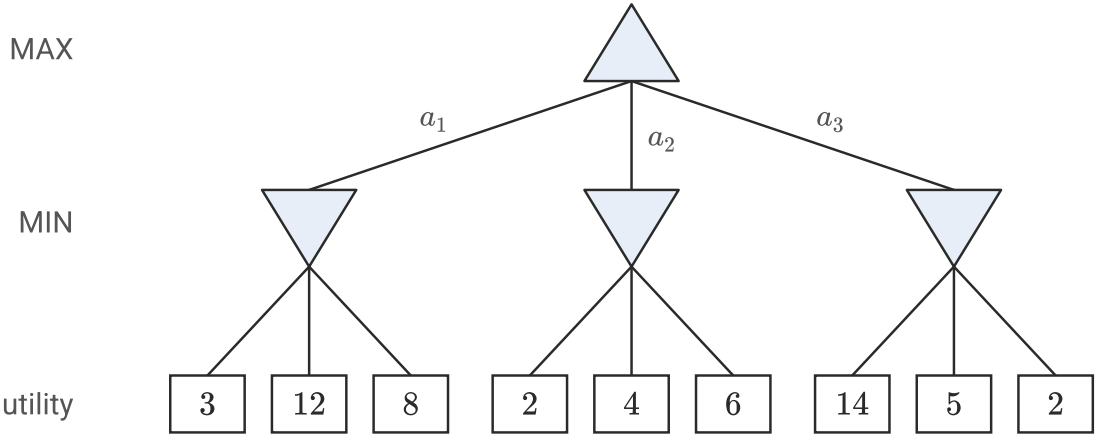


Adversarial search

- In a search problem, the solution $a_{1:T}$ is a fixed plan.
- In a game, MAX cannot know which move MIN will play, **now or later**. It must anticipate **every** reply of MIN, every reply to its own answers, and so on until the game ends.
- A **strategy** π_{MAX} therefore fixes, in advance,
  - its move in the initial state,
  - its move after each possible reply of MIN,
  - its move after each possible reply of MIN to those moves, ...

Perfect play

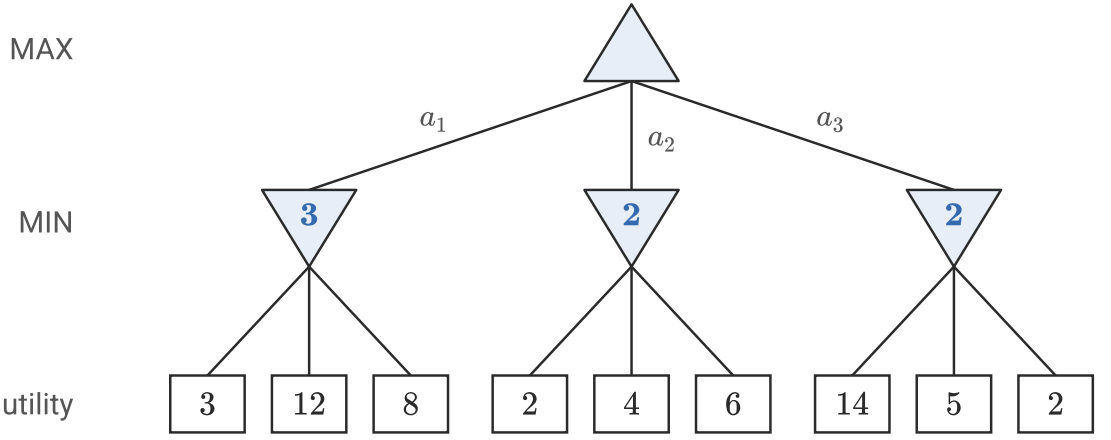
What does MAX obtain from a state if **both players play perfectly** from there on? Reason **backwards**, from the end of the game.



- At the **leaves**, the game is over: the payoff is the utility.

Perfect play

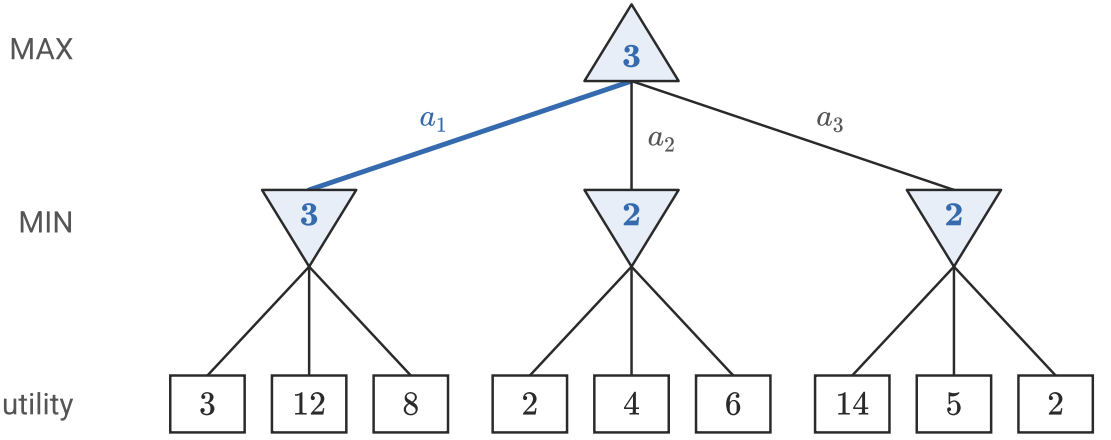
What does MAX obtain from a state if **both players play perfectly** from there on? Reason **backwards**, from the end of the game.



- At the **leaves**, the game is over: the payoff is the utility.
- At a **MIN node**, MIN plays the move that is worst for MAX: the left node is worth $\min(3, 12, 8) = 3$, the other two 2.

Perfect play

What does MAX obtain from a state if **both players play perfectly** from there on? Reason **backwards**, from the end of the game.



- At the **leaves**, the game is over: the payoff is the utility.
- At a **MIN node**, MIN plays the move that is worst for MAX: the left node is worth $\min(3, 12, 8) = 3$, the other two 2.
- At the **root**, MAX plays the move that is best for it, knowing how MIN will answer: $\max(3, 2, 2) = 3$, reached by a_1.

Minimax

The **minimax value** $\text{minimax}(s)$ is the utility that MAX obtains from s when both players play perfectly. Backing up values from the terminal states gives

$$\text{minimax}(s) = \begin{cases} \text{utility}(s) & \text{if } s \in T \\ \max_a \text{minimax}(\text{result}(s,a)) & \text{if MAX plays} \\ \min_a \text{minimax}(\text{result}(s,a)) & \text{if MIN plays,} \end{cases}$$

where the $\max$ and $\min$ are over $a \in \text{actions}(s)$.

The **optimal** next move (for MAX) maximizes the minimax value of the resulting state, $\pi^*(s) = \arg\max_a \text{minimax}(\text{result}(s,a))$.

- $\text{minimax}(s)$ is a **guarantee**: whatever MIN plays, MAX obtains at least $\text{minimax}(s)$, and an optimal MIN holds it to exactly that.
- MAX thus plans for the **worst case**, without any assumption on the strength of the opponent.

Properties of Minimax

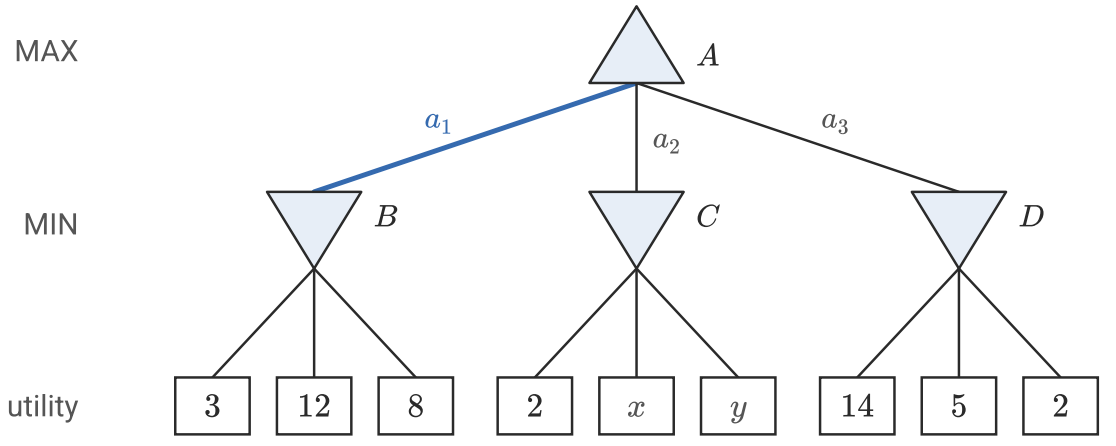
- **Completeness**:
  - Yes, if tree is finite.
- **Optimality**:
  - Yes, if MIN is an optimal opponent.
  - What if MIN is suboptimal?
    - Show that MAX does at least as well.
  - What if MIN is suboptimal and predictable?
    - Other strategies might do better than Minimax. However they may do worse on an optimal opponent.

Minimax efficiency

- Assume $\mathbf{minimax}(s)$ is implemented using its recursive definition.
- How efficient is minimax?
  - Time complexity: same as DFS, i.e., $O(b^m)$.
  - Space complexity:
    - $O(bm)$, if all actions are generated at once, or
    - $O(m)$, if actions are generated one at a time.

> Do we need to explore the whole game tree?

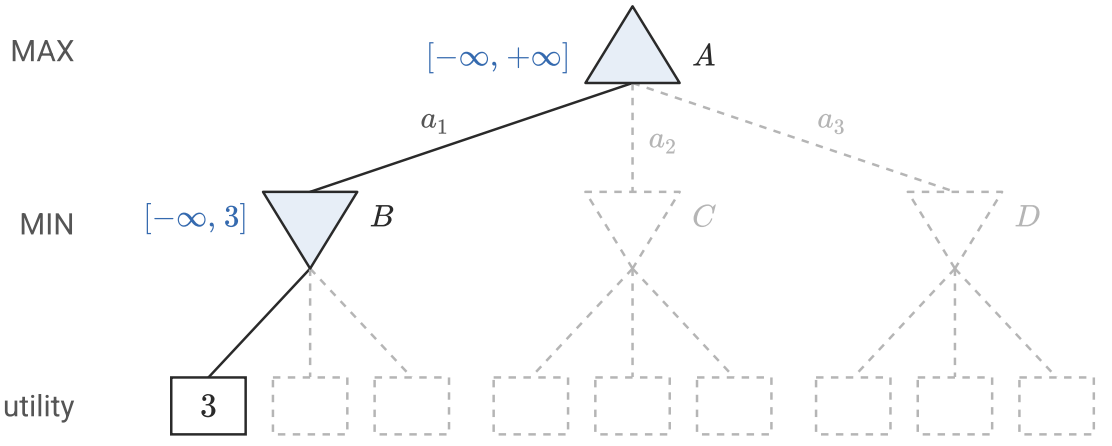
Pruning



$$\begin{aligned}\text{minimax}(\text{root}) &= \max(\min(3,12,8), \min(2,x,y), \min(14,5,2)) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2,x,y) \leq 2 \\ &= 3.\end{aligned}$$

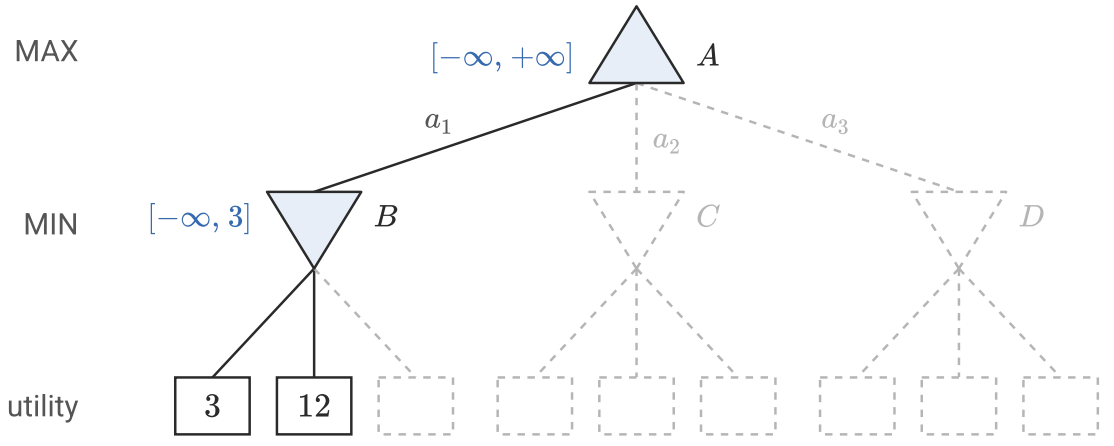
Therefore, it is possible to compute the **correct** minimax decision **without looking at every node** in the tree.

Pruning, step by step



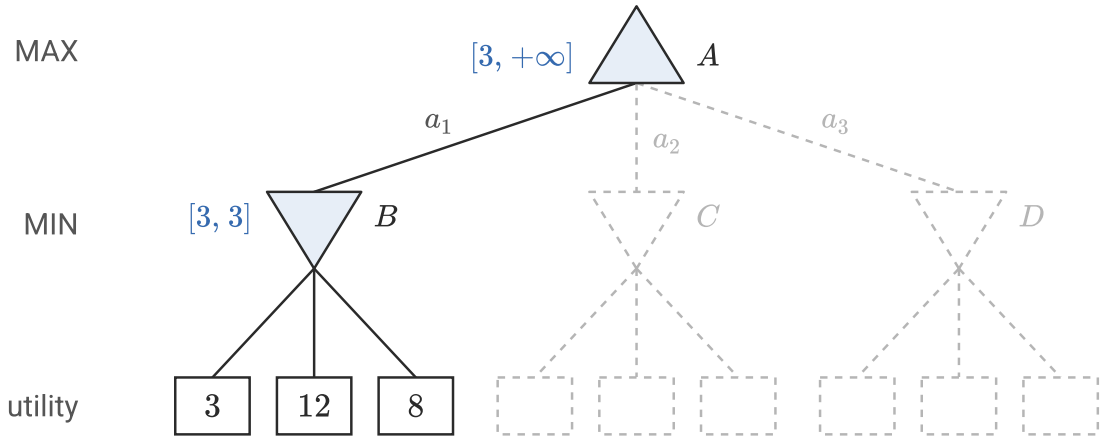
The first leaf of B is 3: B, a MIN node, is worth **at most** 3.

Pruning, step by step



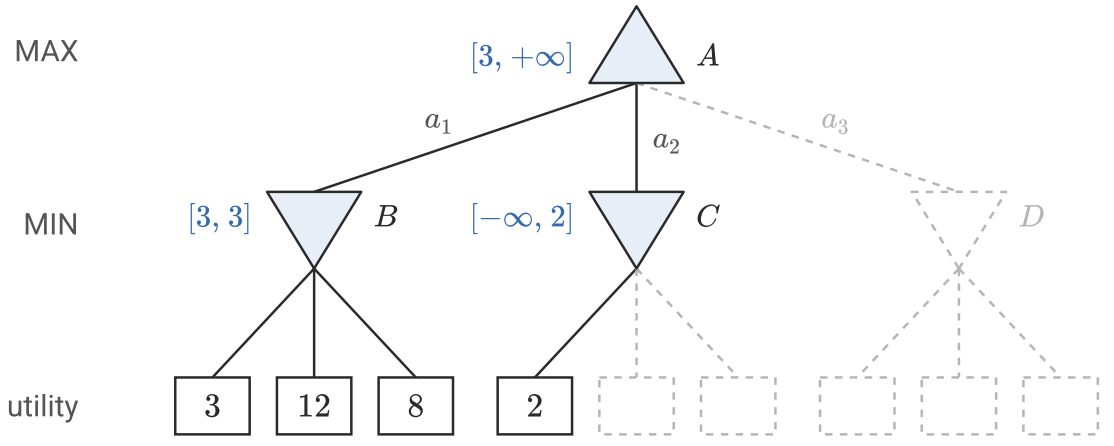
The leaf 12 does not change the bound of B: MIN would not choose it.

Pruning, step by step



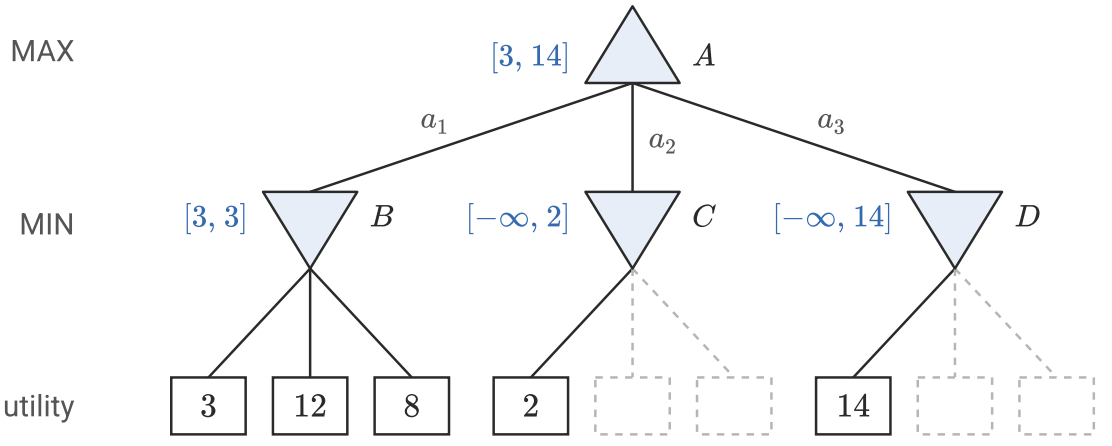
B is worth exactly 3: MAX is guaranteed **at least** 3 at the root.

Pruning, step by step



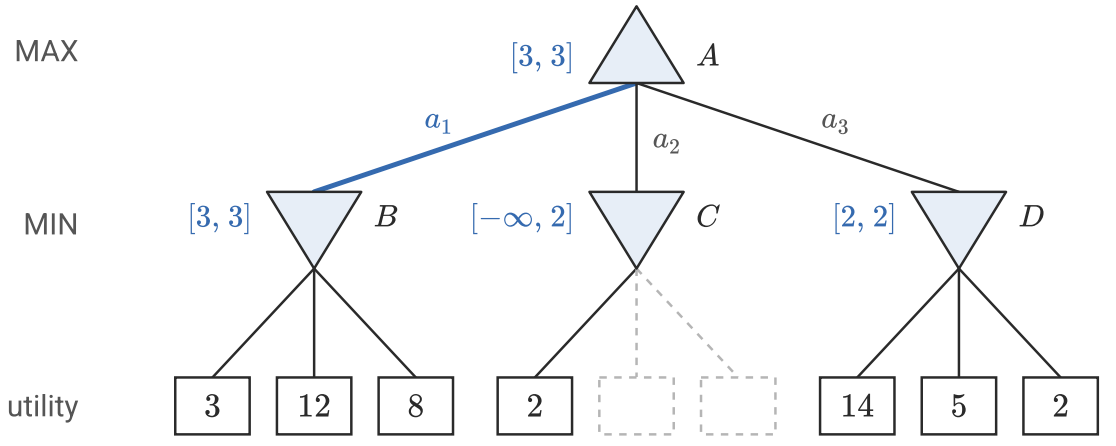
The first leaf of C is 2: C is worth at most $2 < 3$. MAX will never choose C, so its other leaves are **pruned**.

Pruning, step by step



The first leaf of D is 14: D is worth at most 14, and so is the root.

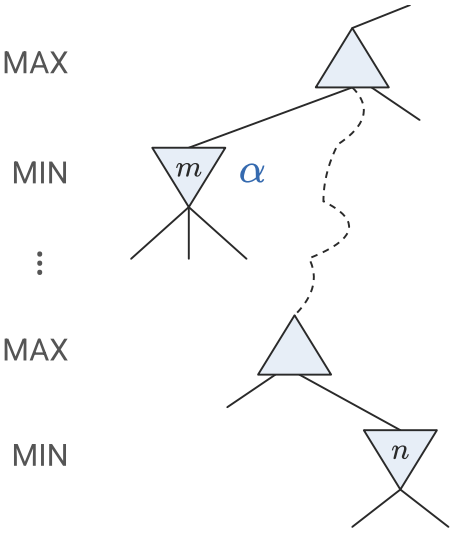
Pruning, step by step



The other leaves bring D down to 2. The root is worth exactly 3, reached by a_1.

We want to compute $v = \mathbf{minimax}(n)$, for $\mathbf{player(n)}$=MIN.

- We loop over n's children.
- The minimax values are being computed one at a time and v is updated iteratively.
- Let α be the best value (i.e., the highest) at any choice point along the path for MAX.
- If v becomes lower than α, then n **will never be reached** in actual play.
- Therefore, we can **stop iterating** over the remaining n's other children.



Similarly, β is defined as the best value (i.e., lowest) at any choice point along the path for MIN. We can halt the expansion of a MAX node as soon as v becomes larger than β.

α-β pruning

- Updates the values of α and β as the path is expanded.
- Prune the remaining branches (i.e., terminate the recursive calls) as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.

α-β search

function ALPHA-BETA-SEARCH($state$) **returns** an action
  $v \leftarrow$ MAX-VALUE($state$, $-\infty$, $+\infty$)
  return the $action$ in ACTIONS($state$) with value v

function MAX-VALUE($state$, α, β) **returns** a utility value
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow -\infty$
  for each a in ACTIONS($state$) **do**
    $v \leftarrow$ MAX(v, MIN-VALUE(RESULT($state$, a), α, β))
    if $v \geq \beta$ **then return** v
    $\alpha \leftarrow$ MAX(α, v)
  return v

function MIN-VALUE($state$, α, β) **returns** a utility value
  if TERMINAL-TEST($state$) **then return** UTILITY($state$)
  $v \leftarrow +\infty$
  for each a in ACTIONS($state$) **do**
    $v \leftarrow$ MIN(v, MAX-VALUE(RESULT($state$, a), α, β))
    if $v \leq \alpha$ **then return** v
    $\beta \leftarrow$ MIN(β, v)
  return v

Properties of α-β search

- Pruning has **no effect** on the minimax values. Therefore, **completeness** and **optimality** are preserved from Minimax.
- Time complexity:
  - The effectiveness depends on the order in which the states are examined.
  - If states could be examined in **perfect order**, then α-β search examines only $O(b^{m/2})$ nodes to pick the best move, vs. $O(b^m)$ for minimax.
    - α-β can solve a tree twice as deep as minimax can in the same amount of time.
    - Equivalent to an effective branching factor $\sqrt{b}$.
- Space complexity: $O(m)$, as for Minimax.

Game tree size



Chess:

- $b \approx 35$ (approximate average branching factor)
- $d \approx 100$ (depth of a game tree for typical games)
- $b^d \approx 35^{100} \approx 10^{154}$.
- For α-β search and perfect ordering, we get $b^{d/2} \approx 35^{50} = 10^{77}$.

Finding the exact solution with Minimax remains **intractable**.

Transposition table

- Repeated states occur frequently because of **transpositions**: distinct permutations of the move sequence end in the same state.
- Like the closed set of graph search, a **transposition table** stores $\text{minimax}(s)$ for every state s already evaluated, so that a repeated state is looked up rather than searched again.

> What data structure should be used to efficiently store and look up the values of states?

Imperfect real-time decisions

Under **time constraints**, searching down to the terminal states is not feasible in most realistic games. The solution is to cut the search earlier:

- replace the $\text{utility}(s)$ function with a heuristic **evaluation function** $\text{eval}(s)$ that estimates the state utility;
- replace the terminal test by a **cutoff test** $\text{cutoff-test}(s, d)$ that decides when to stop expanding a state s at depth d.

$$\text{h-minimax}(s, d) = \begin{cases} \text{eval}(s) & \text{if cutoff} \\ \max\limits_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MAX plays} \\ \min\limits_a \text{h-minimax}(\text{result}(s, a), d + 1) & \text{if MIN plays.} \end{cases}$$

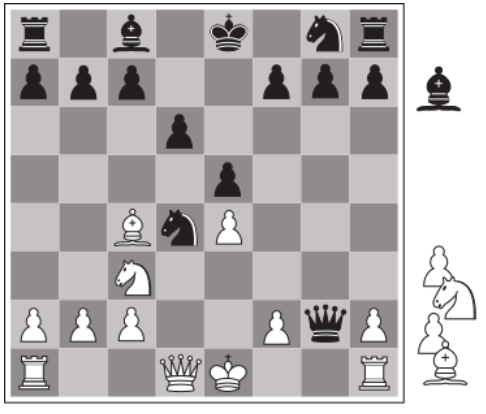
> Can α-β search be adapted to implement H-Minimax?

Evaluation functions

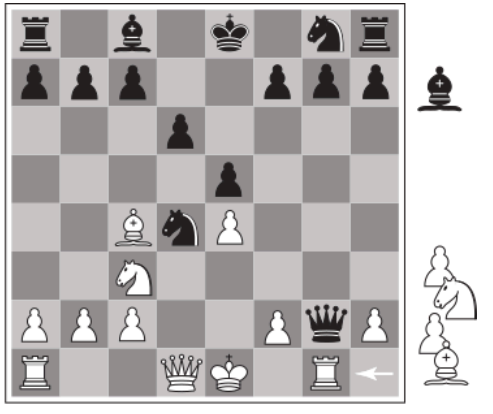
An evaluation function $\mathrm{eval}(s)$ returns an **estimate** of the expected utility of the game from a state s. Its computation **must be short**: that is the whole point of cutting the search.

Ideally, $\mathrm{eval}(s)$ **orders** states as their minimax values do. Its values may differ from the minimax values, as long as the order is preserved. In non-terminal states, it should be strongly **correlated** with the actual chances of winning.

Quiescence



(a) White to move



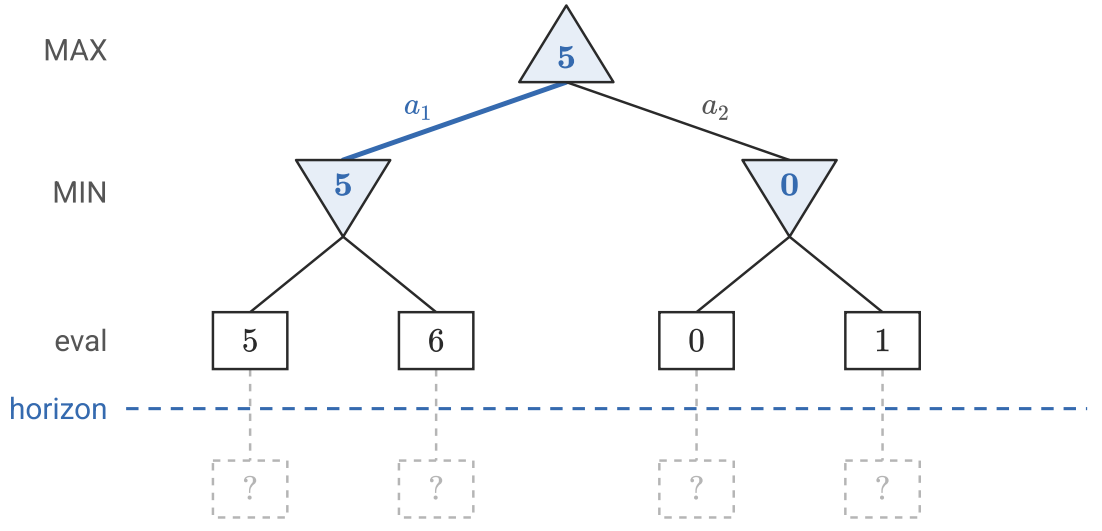
(b) White to move

These two states differ only in the position of the rook at the lower right, yet Black has the advantage in (a) but not in (b). If the search stops in (b), Black does not see that White's next move captures its queen.

The cutoff should only be applied to **quiescent** states, whose value is unlikely to exhibit wild swings in the near future.

The horizon effect

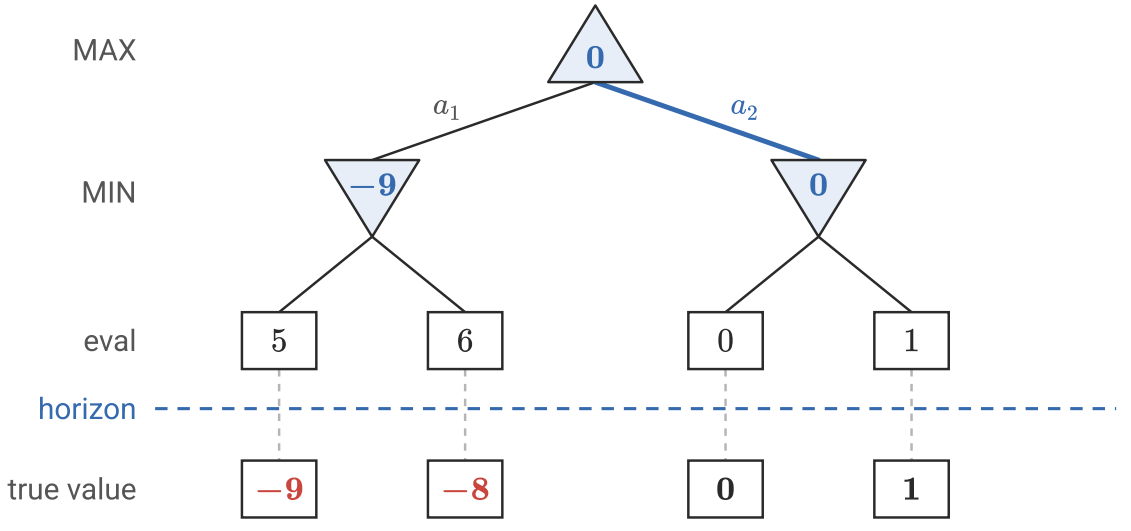
H-Minimax sees the game only down to its cutoff depth, its **horizon**. Below it, $\mathrm{eval}(s)$ stands in for everything that may happen next, and evaluation functions are **always imperfect**.



Searching to depth 2, a_1 looks best: eval rates it 5, against 0 for a_2.

The horizon effect

H-Minimax sees the game only down to its cutoff depth, its **horizon**. Below it, $\mathrm{eval}(s)$ stands in for everything that may happen next, and evaluation functions are **always imperfect**.

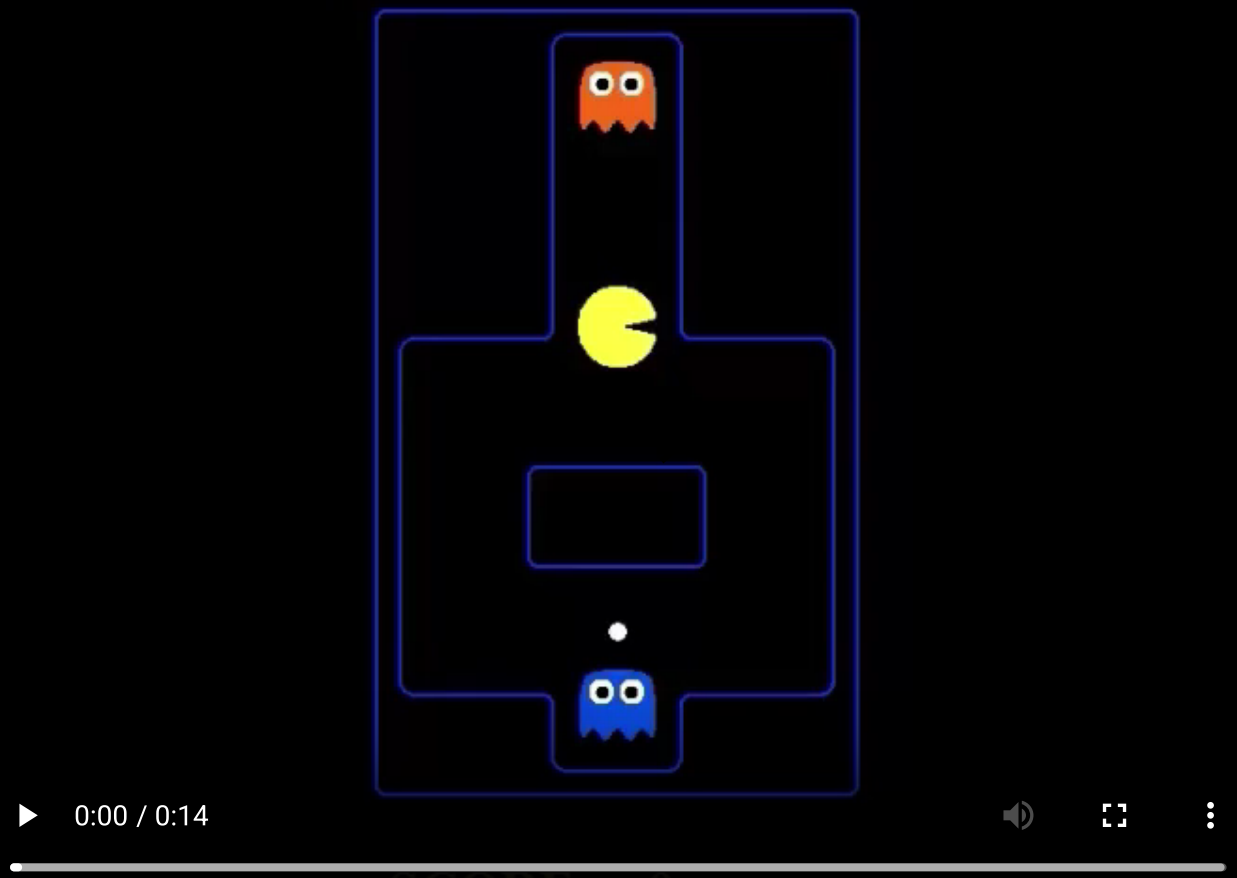


Beyond the horizon, a_1 loses: what it seemed to gain is paid back later (-9). The search took a **bad move** for a **good move**, because its consequences were hidden beyond the horizon.

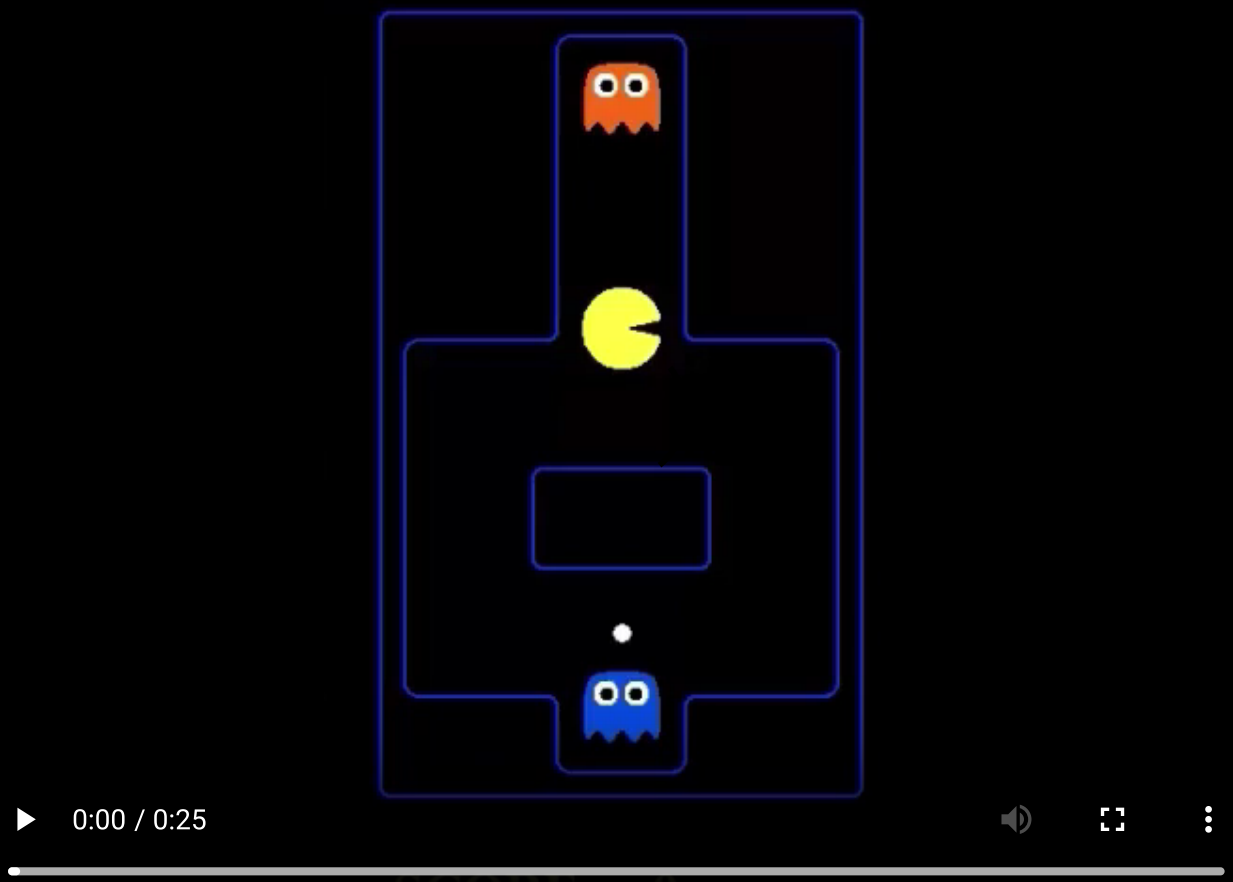
Depth versus evaluation

A **deeper search** compensates an inaccurate eval: the evaluated states are closer to the end of the game, where eval is more accurate. At the limit, the search reaches the terminal states and needs no eval at all.

The converse does not hold: a better eval cannot compensate a **shallow search**, since a perfect one, $\mathrm{eval}(s) = \mathrm{minimax}(s)$, would require solving the game.



Cutoff at depth 2, evaluation = the closer to the dot, the better.



Cutoff at depth 10, evaluation = the closer to the dot, the better.

Multi-agent games

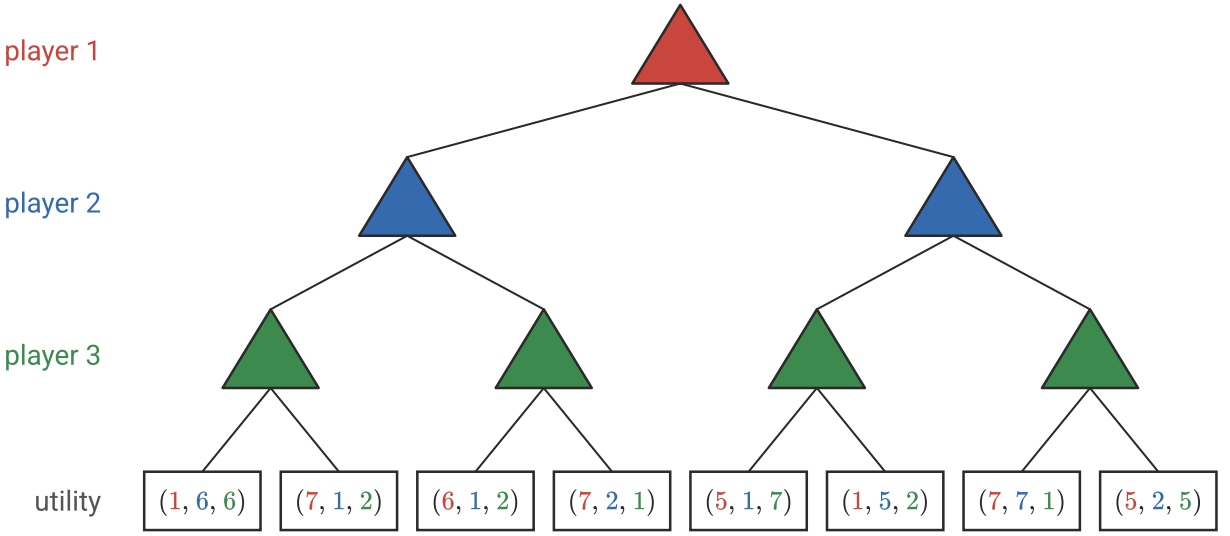
For games that are not zero-sum, or have **more than two players**, minimax generalizes by backing up a **vector** of utilities $v(s) \in \mathbb{R}^N$, one per player, where the player to move maximizes its own component:

$$v(s) = \begin{cases} (\text{utility}(s,1), \ldots, \text{utility}(s,N)) & \text{if } s \in T \\ v(\text{result}(s, a^*)) & \text{otherwise,} \end{cases}$$

where $a^* = \arg\max_a v_{\text{player}(s)}(\text{result}(s,a))$.

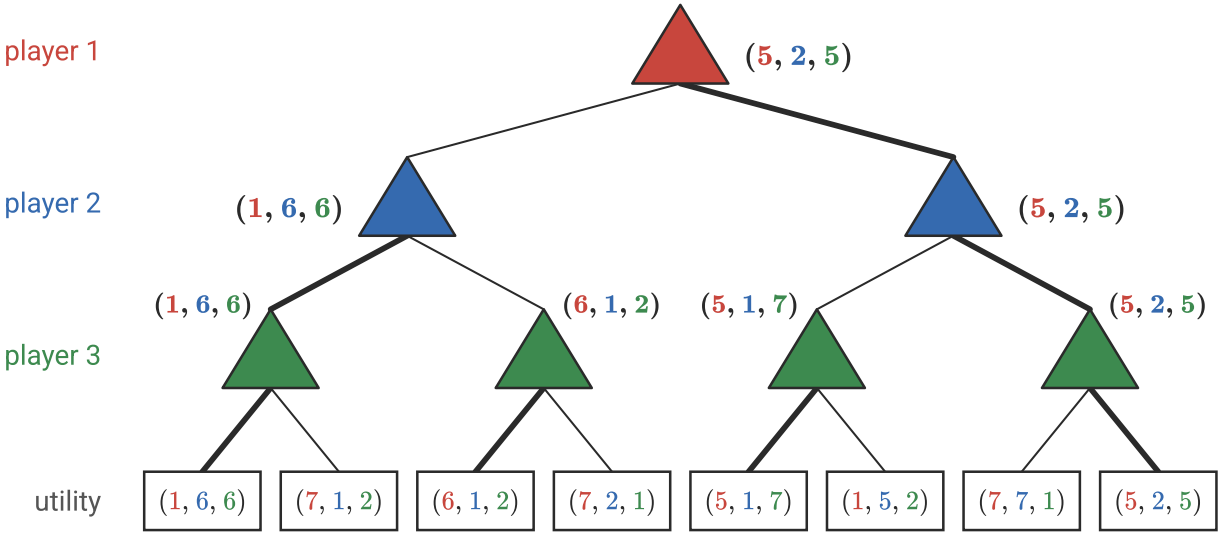
In a two-player zero-sum game, $v_2(s) = c - v_1(s)$, and maximizing v_2 is minimizing v_1: this is minimax.

Cooperation and competition



Without a zero-sum constraint, the interests of the players are no longer opposed: they may **cooperate** or **compete**, depending on the state.

Cooperation and competition

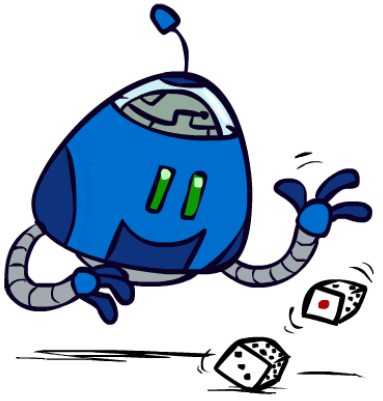


Without a zero-sum constraint, the interests of the players are no longer opposed: they may **cooperate** or **compete**, depending on the state.

Stochastic games

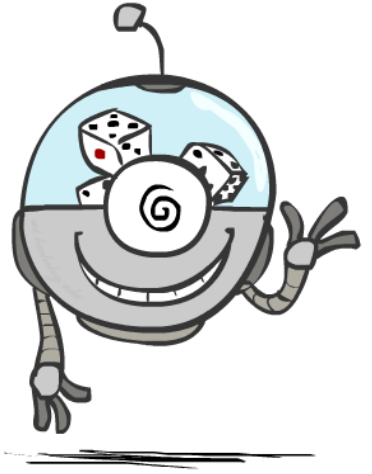
Stochastic games

In real life, unpredictable external events can put an agent into unforeseen situations. Games that mirror this unpredictability are called **stochastic games**: they include a random element, such as explicit randomness (rolling dice) or actions that may fail (the wheels of a robot might slip).

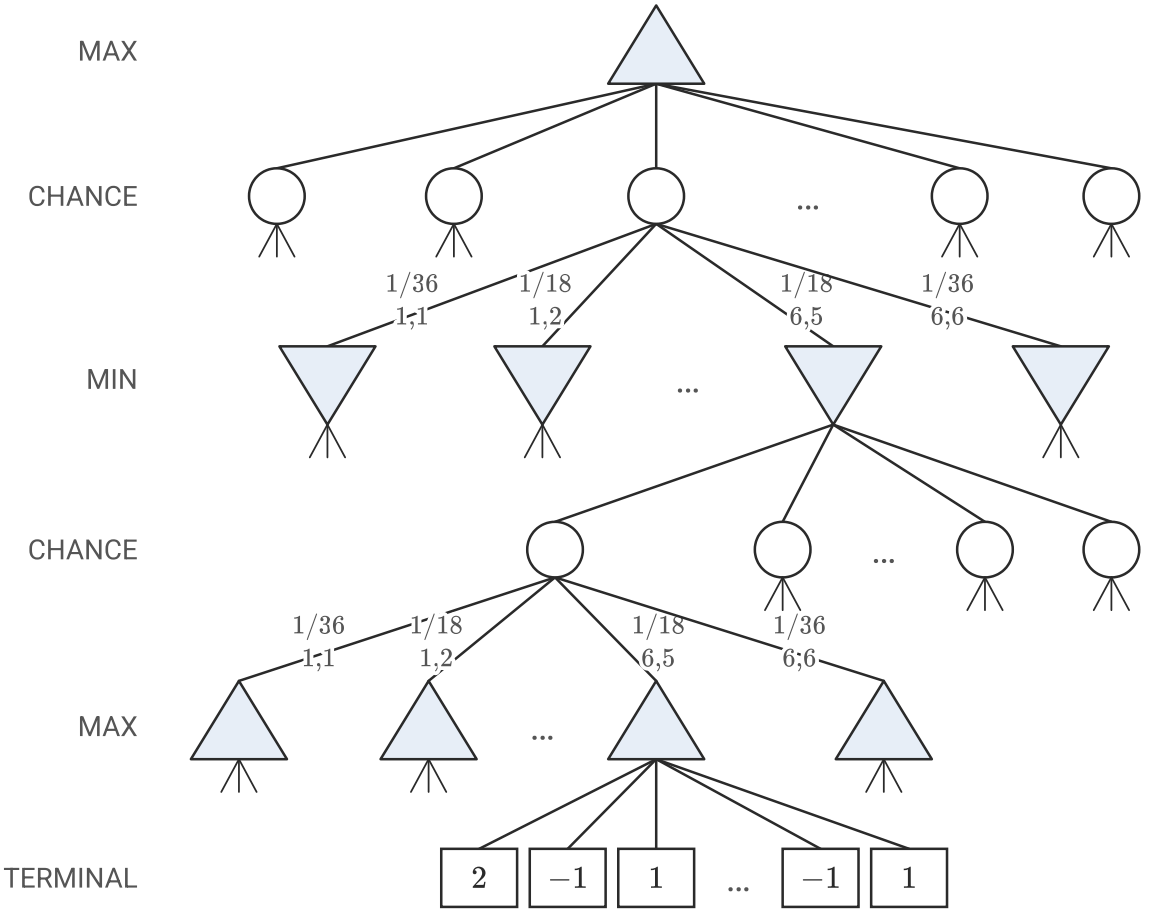


In a game tree, the random element is **modeled** with **chance nodes**. A chance node stands for a state-action pair (s, a), from which the next state s' follows with **probability** $P(s' \mid s, a)$.

This amounts to treating the environment as an extra **random player**, CHANCE, that moves after each of the other players.



Stochastic game tree



Expectiminimax

Because of the uncertainty in the outcomes of actions, states no longer have a **definite** $\mathrm{minimax}$ value.

The value of a chance node (s, a) is instead the **expected** value of the state that follows,

$$q(s, a) = \sum_{s'} P(s' \mid s, a)\, v(s'),$$

the average over the outcomes, weighted by their probabilities, where $\mathrm{minimax}$ would assume the worst outcome. A deterministic action is the special case $P(\mathrm{result}(s, a) \mid s, a) = 1$.

The expectiminimax recursion

Writing $v(s) = \mathbf{expectiminimax}(s)$ for short, the values of states and chance nodes are defined together,

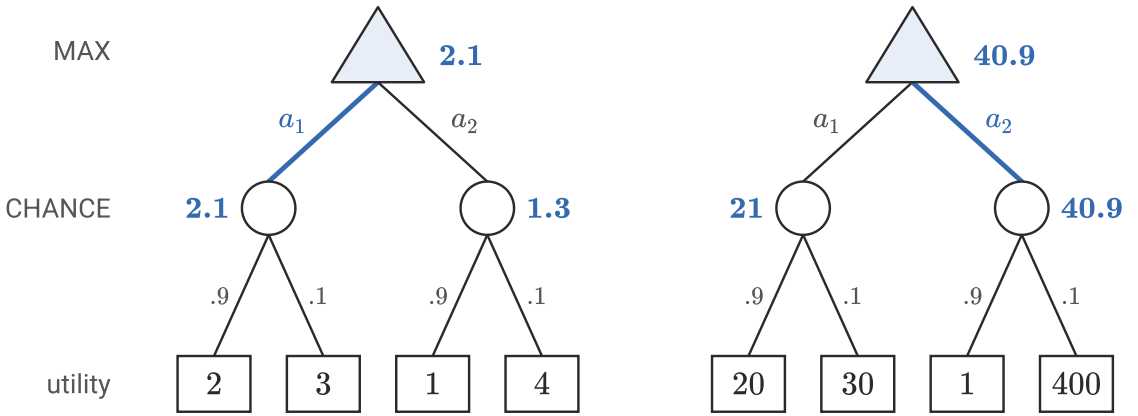
$$v(s) = \begin{cases} \mathrm{utility}(s) & \text{if } s \in T \\ \max\limits_a q(s,a) & \text{if MAX plays} \\ \min\limits_a q(s,a) & \text{if MIN plays,} \end{cases} \qquad q(s,a) = \sum_{s'} P(s' \mid s, a)\, v(s').$$

> Does taking the rational move mean the agent will be successful?

Evaluation functions

As for $\mathrm{minimax}(s)$, $\mathrm{expectiminimax}(s)$ may be approximated by cutting the recursion early and using an evaluation function $\mathrm{eval}(s)$.

With chance nodes, preserving the order of states is **not enough**: expectations add values, so $\mathrm{eval}(s)$ must be a **positive linear transformation** of the expected utility for the move to stay correct.



An order-preserving transformation on leaf values changes the best move.

If the utilities are bounded, α-β search can be adapted to stochastic games.

Monte Carlo tree search

Random playout evaluation

To evaluate a state s, let the algorithm play **against itself** from s with **random moves**, thousands of times. Each such game is a **random playout**, and the evaluation is the proportion of playouts won,

$$\text{eval}(s) = \frac{1}{K} \sum_{k=1}^{K} \mathbf{1}[\text{playout } k \text{ is won}].$$

This evaluation requires **no domain knowledge**: the game engine is all that is needed.

Monte Carlo Tree Search

Evaluating every move with the same number of random playouts wastes most of them on bad moves. MCTS instead grows a search tree from the root and spends its playouts on the **most promising moves**.

Each node n in the current search tree maintains two values:

- the number of wins $Q(n, p)$ of each player p, over the playouts that passed through n;
- the number $N(n)$ of times n has been visited.

Each round of the search does four things: **selection** down to a node n that is not fully expanded, **expansion** of one child n' of n, **simulation** of a random playout from n', and **backpropagation** of the result along the path back to the root.

function MCTS($state$, $budget$) **returns** an action
  $root \leftarrow$ MAKE-NODE($state$)
  while time remains in $budget$ **do**
    $n \leftarrow$ SELECT($root$)
    $n' \leftarrow$ EXPAND(n)
    $reward \leftarrow$ SIMULATE(n')
    BACKPROPAGATE(n', $reward$)
  end
  return the action leading to the child of $root$ with the largest N

function SELECT(n) **returns** a node to expand
  while n is fully expanded and not terminal **do**
    $n \leftarrow$ the child n' of n maximising UCB1
  end
  return n

Rounds are repeated for as long as the time budget allows. The move played is the one leading to the most visited child of the root.

Exploration and exploitation

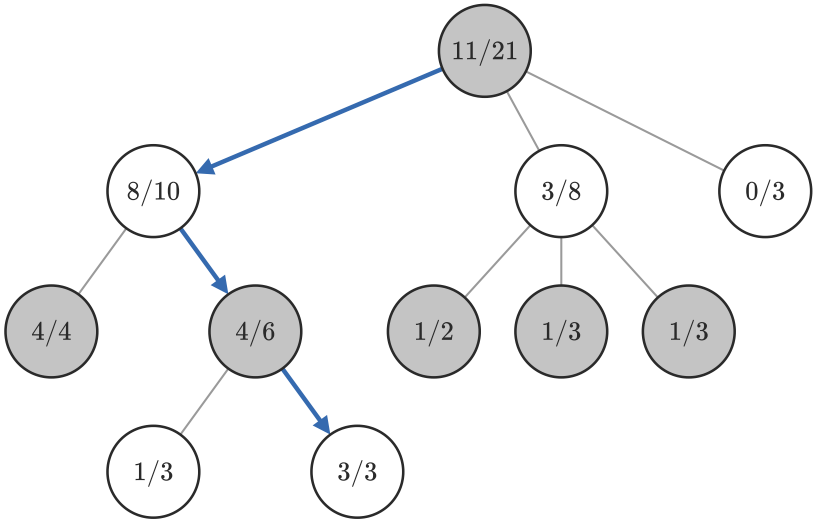
Given a limited budget of random playouts, the efficiency of MCTS critically depends on the choice of the nodes made during the **selection** step.

During the selection step, the UCB1 policy picks the child node n' of n that maximizes

$$\frac{Q(n', p)}{N(n')} + c\sqrt{\frac{\log N(n)}{N(n')}},$$

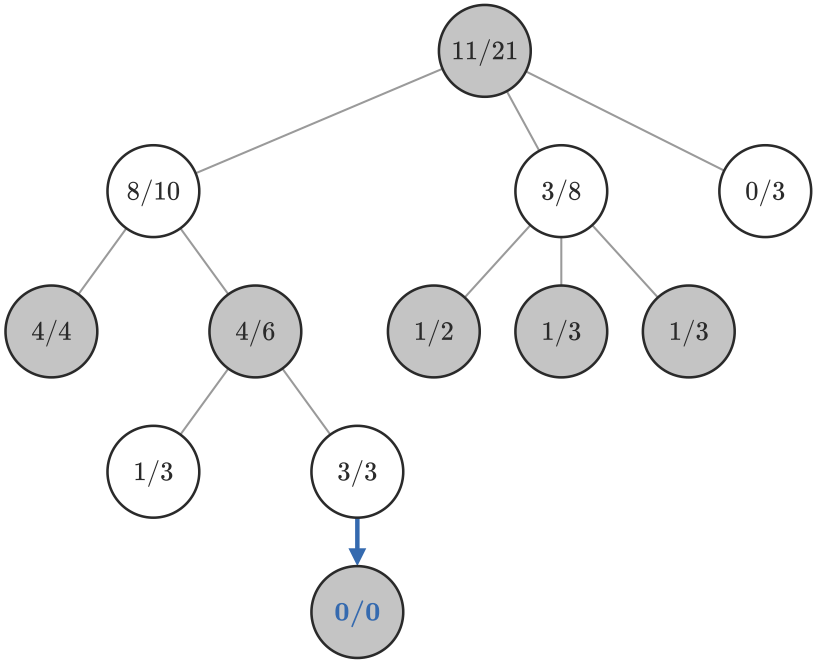
where p is the player to move in n. The first term encourages the **exploitation** of nodes with a high win rate, the second the **exploration** of nodes visited little, and the constant $c > 0$ sets the trade-off between the two.

One round of MCTS



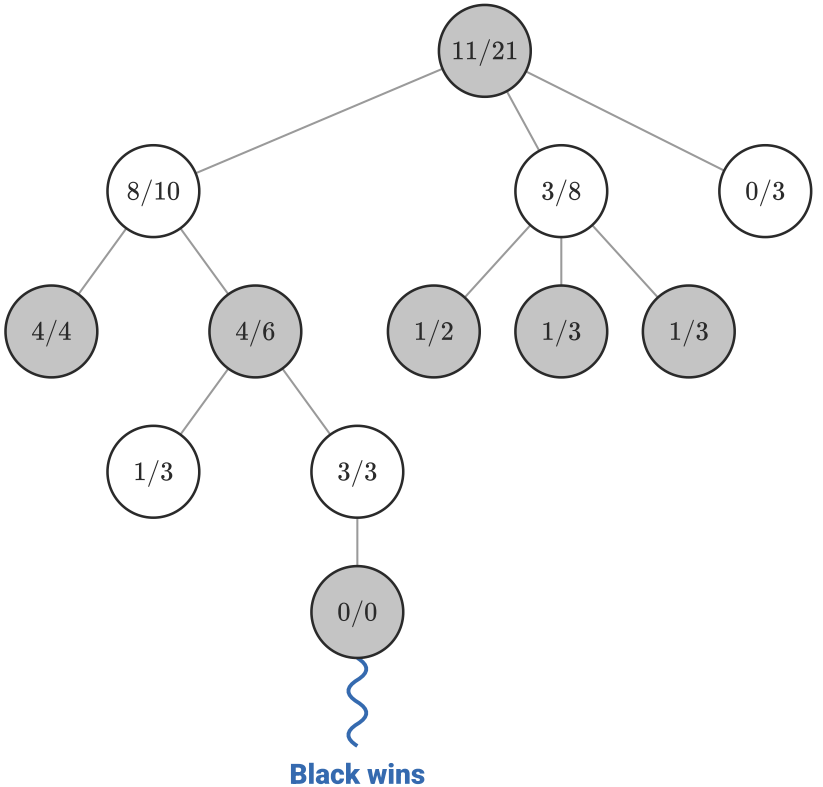
Grey nodes are Black's turns, white nodes White's; each shows Black's wins and visits. **Selection**: from the root, follow UCB1 down to a node that is not fully expanded.

One round of MCTS



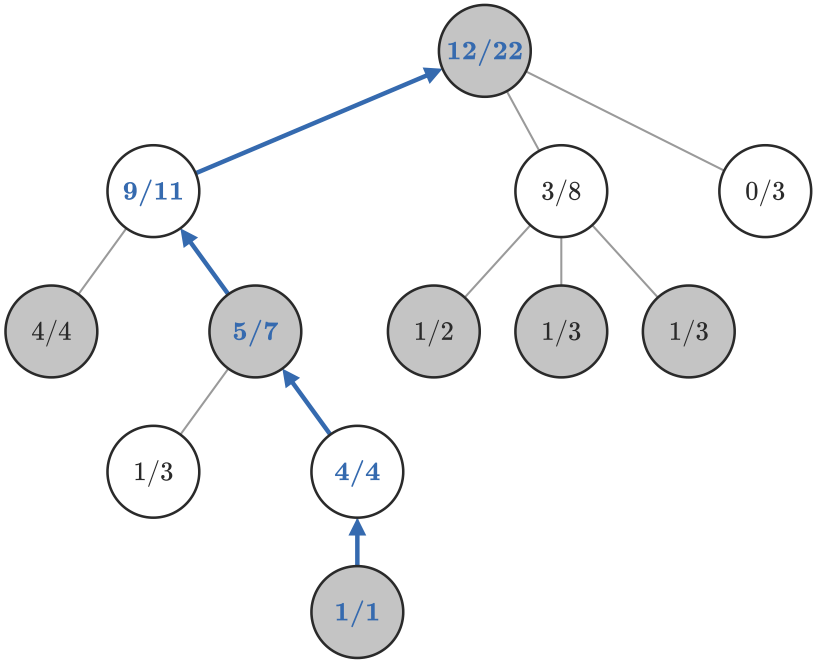
Expansion: add one of its children, not visited yet (0/0).

One round of MCTS



Simulation: play a random playout from the new node. Here, Black wins.

One round of MCTS



Backpropagation: update the counts along the path back to the root: one more visit for every node and, since Black won, one more win for Black.

Modeling assumptions

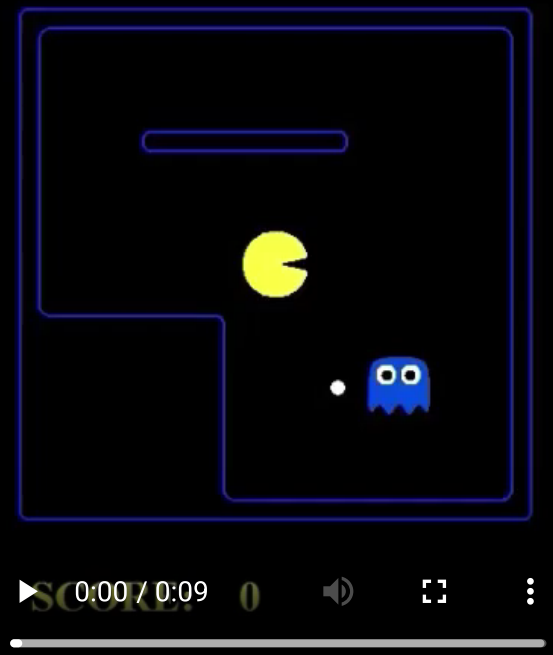


What if our assumptions are incorrect?

Setup

- P_1: Pacman uses depth 4 search with an evaluation function that avoids trouble, while assuming that the ghost follows P_2.
- P_2: Ghost uses depth 2 search with an evaluation function that seeks Pacman, while assuming that Pacman follows P_1.
- P_3: Pacman uses depth 4 search with an evaluation function that avoids trouble, while assuming that the ghost follows P_4.
- P_4: Ghost makes random moves.





Minimax Pacman (P_1) vs. Adversarial ghost (P_2)

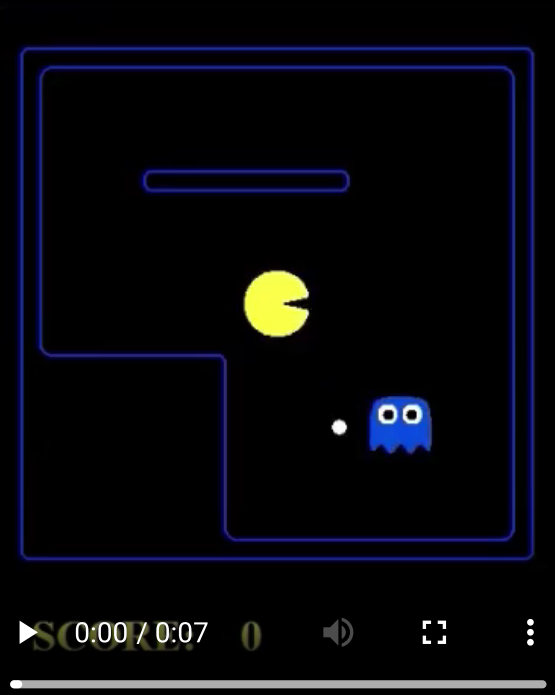


Minimax Pacman (P_1) vs. Random ghost (P_4)

Credits: CS188, UC Berkeley.



Expectiminimax Pacman (P_3) vs. Random ghost (P_4)



Expectiminimax Pacman (P_3) vs. Adversarial ghost (P_2)

State-of-the-art game programs

Perfect information



- 1997, **Deep Blue** defeats Garry Kasparov at chess: 2×10^8 positions per second, a sophisticated evaluation function, and search extended up to 40 plies on some lines.
- 2007, **checkers is solved**: 10^{14} calculations over 18 years prove that perfect play by both sides is a draw.
- 2016, **AlphaGo** beats Lee Sedol 4-1, then Ke Jie in 2017, by combining Monte Carlo tree search with deep learning trained on human and self-play games.

Chess and Go remain unsolved: only their best play is now superhuman.

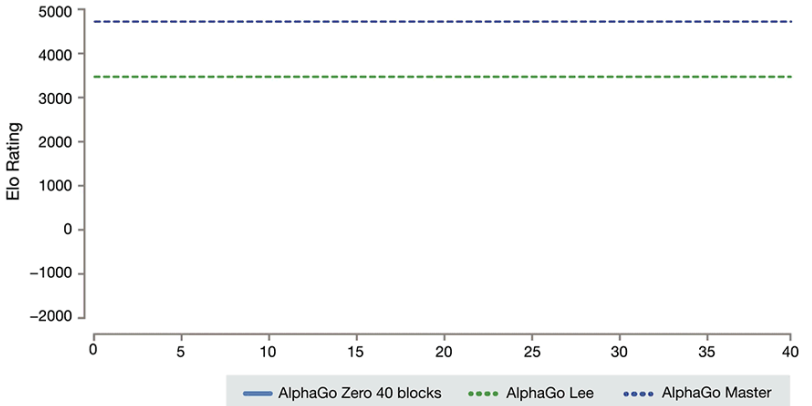
Schaeffer et al., "Checkers is solved", Science, 2007; Silver et al., "Mastering the game of Go", Nature, 2016.



Press coverage for the victory of AlphaGo against Lee Sedol.

Learning to play, from less and less

- 2017, **AlphaGo Zero** reaches the same level from self-play only, with no human games.
- 2018, **AlphaZero** applies the same recipe to chess, shogi and Go, given only the rules.
- 2020, **MuZero** drops the rules too: it learns a model of the game and plans with it, mastering Go, chess, shogi and Atari.



Credits: AlphaGo Zero: Learning from scratch; Schrittwieser et al., "Mastering Atari, Go, chess and shogi by planning with a learned model", Nature, 2020.

Imperfect information

The assumption of perfect information we made at the start is what these games drop:

- 2017-2019, **Libratus** then **Pluribus** beat professionals at no-limit poker, the latter with six players at the table, where no notion of optimal play against all opponents exists.
- 2022, **DeepNash** reaches expert level at Stratego, where each side hides its pieces.
- 2022, **Cicero** plays Diplomacy at human level, combining planning with negotiation in natural language.

> What does an optimal strategy even mean when the opponents may cooperate, and lie?

Brown and Sandholm, Science, 2017 and 2019; Perolat et al., Science, 2022; Meta FAIR, Science, 2022.

Summary

- A **game** is defined by its states, actions and transitions, a function $\mathrm{player}(s)$, terminal states $T \subseteq \mathcal{S}$ and a utility $\mathrm{utility}(s, p)$. Its solution is a strategy $\pi_p : \mathcal{S} \to \mathcal{A}$, not a sequence $a_{1:T}$.
- **Minimax** backs the utilities up the tree, $\max_a$ for MAX and $\min_a$ for MIN, in $O(b^m)$ time and $O(bm)$ space.
- α-β **pruning** returns the same move while examining $O(b^{m/2})$ nodes at best: twice the depth in the same time.
- **H-minimax** cuts the search at depth d and replaces the utility by an evaluation function, at the price of the horizon effect.
- **Expectiminimax** averages over chance nodes, $q(s, a) = \sum_{s'} P(s' \mid s, a)\, v(s')$, and **Monte Carlo tree search** samples random playouts instead of evaluating.
- Each is optimal only **relative to a model** of the opponent. Assuming the wrong one is not playing well.

The end.